SECURITY EAGLES

CRTP

CERTIFIED RED TEAM PROFESSIONAL

PART III

ACTIVE DIRECTORY ENUMERATION

www.securityeagles.com

FROM LEARNERS TO LEADERS

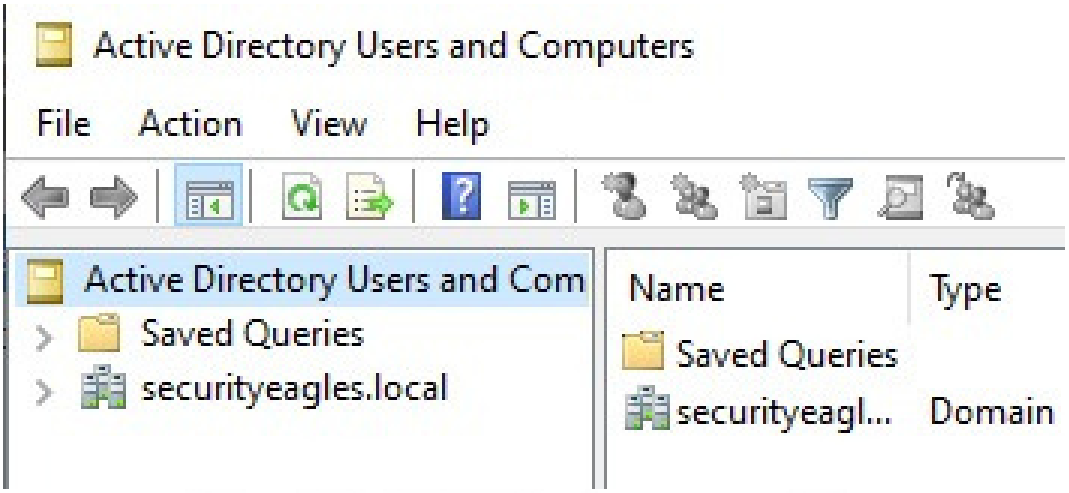
Before we start

- We will begin with a foundational lab that introduces the core concepts.
- This lab environment is designed to be scalable and modular, allowing us to continuously expand and add complexity to the exercises as the course progresses and new topics are introduced.

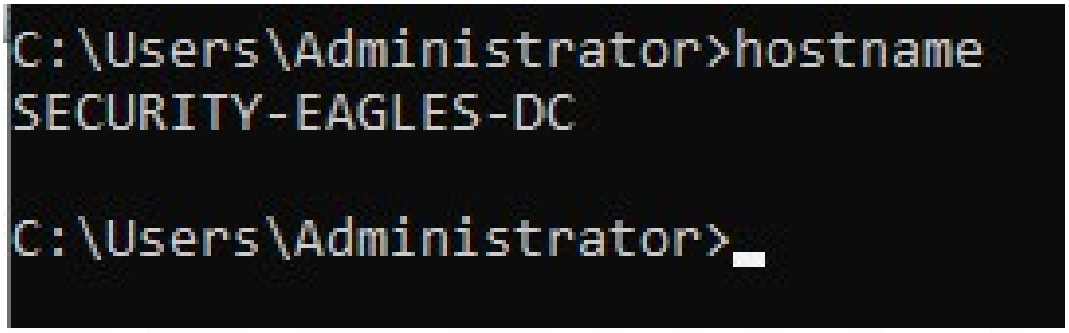
Home Lab requirements (What you'll need)

- Virtualization Platform (Hypervisor):
  A stable platform to run your virtual machines, such as VMware Workstation/Fusion or Oracle VirtualBox.
- Windows Server Virtual Machine (VM):
  This machine will be configured as your Domain Controller (DC), the primary target for Active Directory attacks
  - Windows Server (2022 or 2025 recommended)
- Windows Workstation Virtual Machine (VM):
  this will serve as a target host and a foothold host within the virtual domain, mimicking a typical corporate machine.
  - Windows 10 or Windows 11
- A linux attacking machine in the same network , like kali linux

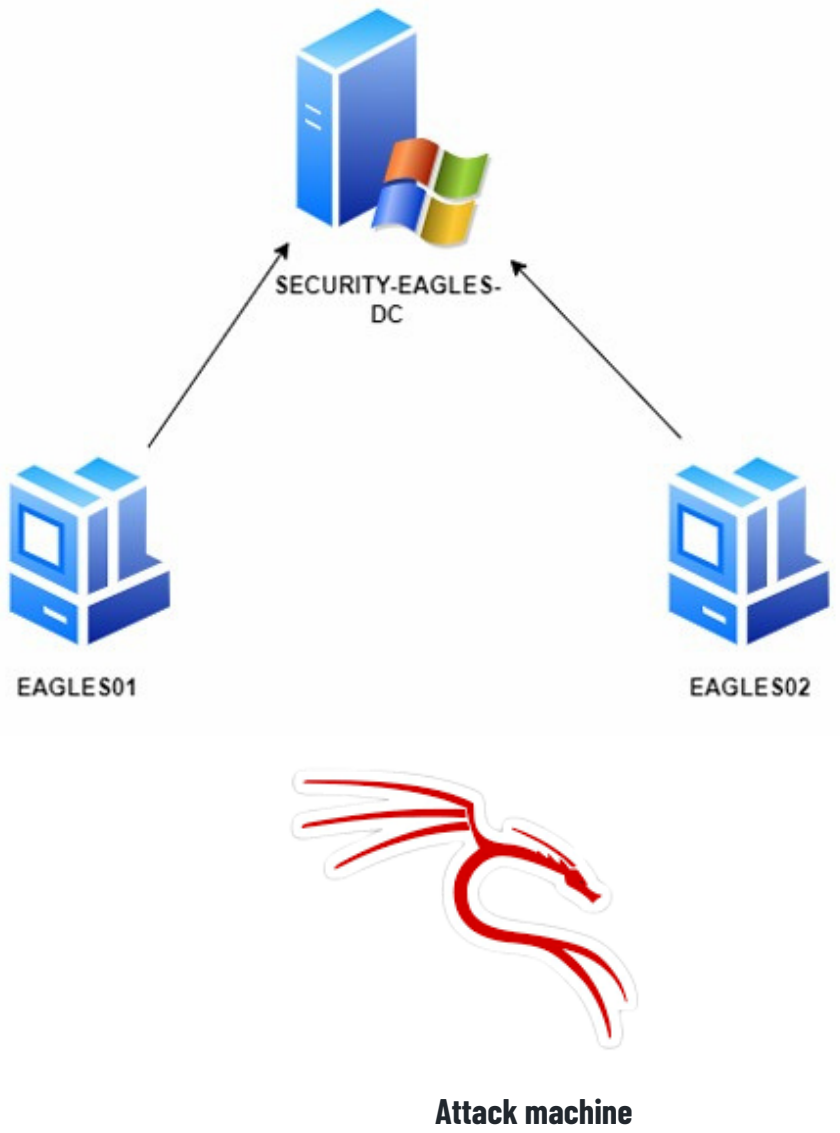
For the domain name we are going to use securityeagles.local to follow along this course

- 

- This will be the hostname for the Domain Controller.



- Our lab architecture consists of three machines: the Domain Controller, named SECURITY-EAGLES-DC, and two workstations, EAGLES01 and EAGLES02, all joined to the securityeagles.local domain .



3. ACTIVE DIRECTORY ENUMERATION

- Domain discovery & topology Users & groups
- enumeration Computers enumeration
- Forest/intra-forest trust analysis
- GPO discovery & settings audit ACL
- inspection & interesting rights Service, SPN &
- delegation discovery Credential sources
- (configs, secrets) Session & logon
- enumeration BloodHound mapping & queries
- Automation & scripting (PowerShell/C#)

- Having mastered the foundational enumeration techniques in the previous section, we must now expand our toolkit to access richer data
- In this section, we will continue to utilize PowerView and will also introduce the native ActiveDirectory PowerShell Module.
- Unlike PowerView, which is a third-party script, the ActiveDirectory module is a Microsoft-native feature, though it is often not installed by default on standard workstations . Therefore, our initial step must be to ensure the module is locally available.
- The following is the command required to download and load the ActiveDirectory module in our Windows environment.

```
Get-WindowsCapability -Name RSAT.ActiveDirectory* -Online |
Add-WindowsCapability -Online
```

```
PS C:\Users\m.dynamo> Get-WindowsCapability -Name RSAT.ActiveDirectory* -Online | Add-WindowsCapability -Online


Path          :
Online        : True
RestartNeeded : False
```

- Executing this command will install the necessary components, granting us access to powerful cmdlets like Get-ADUser and Get-ADGroup

- Once the module is successfully installed, we import it into our current session using the Import-Module command.

```
Import-Module ActiveDiretory
```

```
PS C:\Users\m.dynamo\Downloads> Import-Module ActiveDirectory
PS C:\Users\m.dynamo\Downloads>
```

- Confirm the successful import by checking for available cmdlets like Get-ADDomain or Get-ADUser
- With the module now loaded, we can begin leveraging its native capabilities for reconnaissance, starting with basic domain information .

1- Domain discovery & topology

- PowerHuntShares is PowerShell tool designed to help cybersecurity teams and penetration testers better identify, understand, attack, and remediate SMB shares in the Active Directory environments they protect
- To initiate its usage, we begin by importing the PowerHuntShares.psm1 module into our current PowerShell session (using the provided source file).

```
Import-Module .\PowerHuntShares.psm1
```

```
PS C:\Users\m.dynamo\Downloads\PowerHuntShares-main> Import-Module .\PowerHuntShares.psm1
PS C:\Users\m.dynamo\Downloads\PowerHuntShares-main>
```

- Once imported, the primary cmdlet for reconnaissance will be active.

source : https://github.com/NetSPI/PowerHuntShares

1- Domain discovery & topology

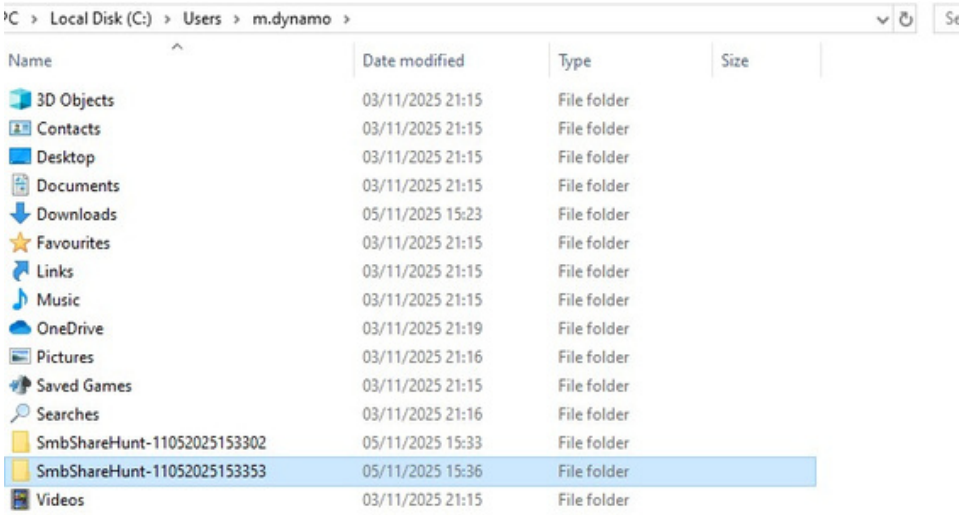
- PowerHuntShares is highly configurable, offering options to specify targets, ports, or filtering criteria.
- While the tool offers numerous parameters for detailed filtering, we will concentrate on the features that provide the highest offensive impact and rapid identification of critical vulnerabilities.
- We specify the IP address of the Domain Controller, along with a valid domain username and corresponding password, as demonstrated in the following command.

```
Invoke-HuntSMBShares -NoPing -OutputDirectory
C:\Users\m.dynamo\ -DomainController 192.168.100.34 -Username
SECURITYEAGLES\m.dynamo -Password “GreatADLab123”
```

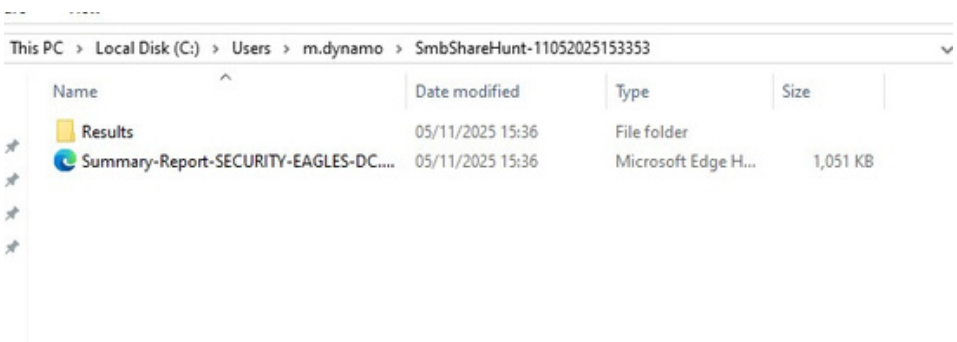
```
PS C:\Users\m.dynamo\Downloads\PowerHuntShares-main> Invoke-HuntSMBShares -NoPing -OutputDirectory C:\Users\m.dynamo\ -DomainContro
ller 192.168.100.34 -Username SECURITYEAGLES\m.dynamo -Password "GreatADLab123"
```

- Successfully executing this command authenticates our session to the Domain Controller and returns the expected output

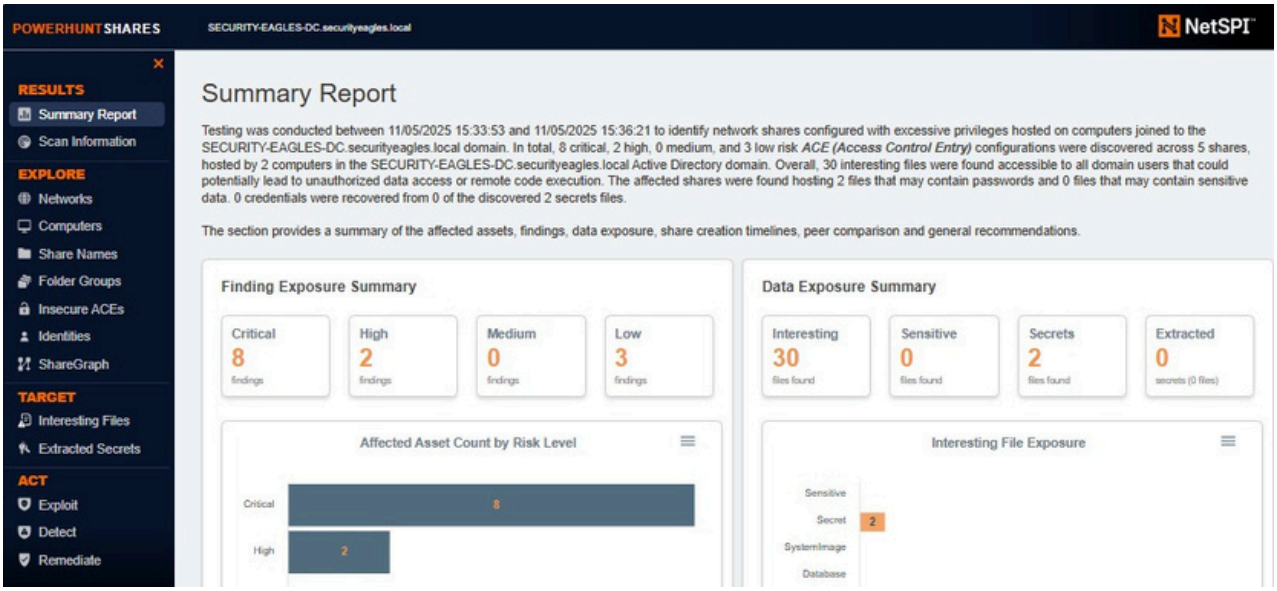
- The tool generates a detailed report stored in a dedicated folder named using the pattern SmbShareHunt-<timestamp>, which encapsulates all the reconnaissance output.



- Inside this output directory, you will find a summary report formatted in HTML, which provides a high-level overview of the scanned environment.

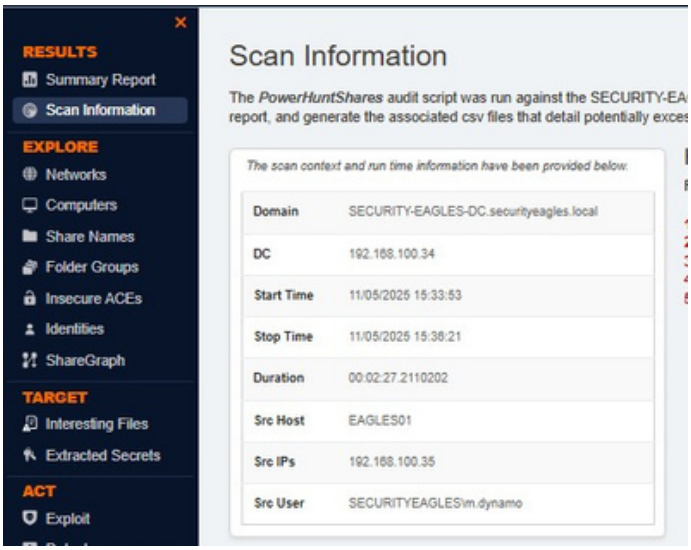


- The primary value of the HTML summary is its ability to immediately categorize and prioritize the findings.
- Double-clicking the report opens an extensive dashboard that displays potential findings categorized by their ranking (Critical, High, Medium, Low).



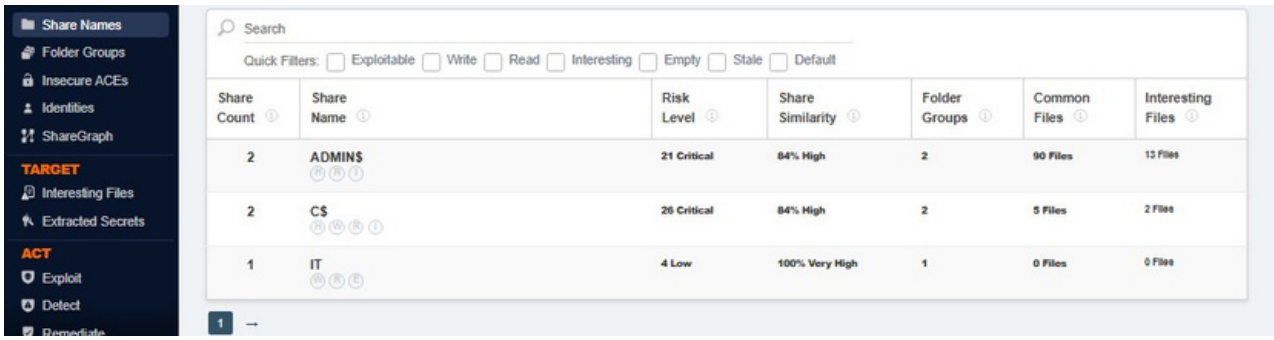
- This ranking system is essential for any professional engagement, as it guides our next steps by highlighting the easiest and most impactful targets.

- Before we analyze the high-risk findings, the report offers a convenient overview of the reconnaissance session itself
- The left-hand panel of the report displays key scan information, revealing important data about the domain such as the Domain Controller's (DC) IP address, the Source Host, and the Source IP addresses.



- This contextual information is essential for documentation, confirming the attacking machine's identity and the targeted Domain Controller.

- Finally, the tool lists all successfully enumerated shares, which can be viewed by clicking the 'Share Names' button.



- Reviewing this full list is essential to ensure no accessible shares were overlooked, regardless of their security ranking.

4- ACL inspection & interesting rights

- Before we can execute domain-specific cmdlets or tools requiring authentication, we must package the user's credentials into a format that PowerShell can securely utilize
- We begin by creating a PSCredential object using the domain user's username and corresponding password.

```
$SecPassword = ConvertTo-SecureString ‘GreatADLab123’ -
AsPlainText -Force
$Cred = New-Object
System.Management.Automation.PSCredential(‘SECURITYEAGLES\m.dyn
amo’, $SecPassword)
```

```
PS C:\Users\m.dynamo\Downloads> $SecPassword = ConvertTo-SecureString 'GreatADLab123' -AsPlainText -Force
PS C:\Users\m.dynamo\Downloads> $Cred = New-Object System.Management.Automation.PSCredential('SECURITYEAGLES\m.dynamo', $SecPassword)
```

- Once the PSCredential object is stored in a variable, it can be passed to tools like PowerView or other native Active Directory cmdlets using the -Credential parameter

4- ACL inspection & interesting rights

- A critical part of ActiveDirectory enumeration is understanding the permissions that govern domain objects, as misconfigurations here often lead to privilege escalation.
- The Get-DomainObjectAcl command helps us to enumerate the Access Control Lists (ACLs) for any domain object.

```
Get-DomainObjectAcl -SamAccountName m.dynamo -credential $Cred
```

```
PS C:\Users\m.dynamo\Downloads> Get-DomainObjectAcl -SamAccountName m.dynamo   -credential $Cred


ObjectDN                : CN=msf dynamo,CN=Users,DC=securityeagles,DC=local
ObjectSID               : S-1-5-21-204598522-1414839267-3477406029-1107
ActiveDirectoryRights   : ReadProperty
ObjectAceFlags          : ObjectAceTypePresent, InheritedObjectAceTypePresent
ObjectAceType           : 4c164200-20c0-11d0-a768-00aa006e0529
InheritedObjectAceType  : 4828cc14-1437-45bc-9b07-ad6f015e5f28
BinaryLength            : 60
AceQualifier            : AccessAllowed
IsCallback              : False
OpaqueLength            : 0
AccessMask              : 16
SecurityIdentifier      : S-1-5-32-554
AceType                 : AccessAllowedObject
AceFlags                : None
IsInherited             : False
```

- In this specific example, we enumerated the ACLs of a user object by specifying its SamAccountName, while also including the -Credential parameter for authentication.
- Note the specific GUID value associated with the ObjectAceType attribute.

- Understanding this GUID is vital, as it uniquely identifies the property being controlled, allowing us to determine if the permission is truly a high-value misconfiguration.
- The raw GUID value currently displayed in the ObjectAceType is difficult to interpret and does not immediately reveal the underlying permission name. To properly analyze what access has been granted, we must translate this identifier.
- We will first set a PowerShell variable named $guid and assign it the retrieved GUID value.
- This intermediate step is necessary for converting the cryptic identifier into a human-readable format.

```
$guid = "4c164200-20c0-11d0-a768-00aa006e0529"
```

```
PS C:\Users\m.dynamo\Downloads> $guid = "4c164200-20c0-11d0-a768-00aa006e0529"
PS C:\Users\m.dynamo\Downloads>
```

- We will now execute the command that uses this variable to map the GUID back to its clear, human-readable property name.

- By utilizing the Get-ADObject cmdlet from the ActiveDirectory module, we can successfully convert the raw GUID value into a human-readable property name.

```
Get-ADObject -SearchBase "CN=Extended-Rights,$((Get-ADRootDSE).ConfigurationNamingContext)" -Filter {ObjectClass -like 'ControlAccessRight'} -Properties * | Select Name,DisplayName,DistinguishedName,rightsGuid | ?{$_.rightsGuid -eq $guid} | fl
```

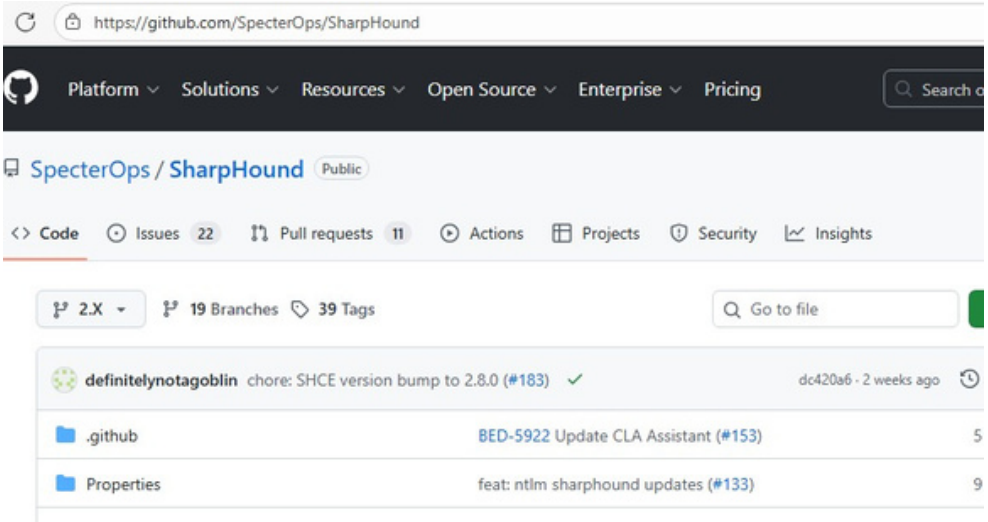
```
PS C:\Users\m.dynamo> Get-ADObject -SearchBase "CN=Extended-Rights,$((Get-ADRootDSE).ConfigurationNamingContext)" -Filter {ObjectClass -like 'ControlAccessRight'} -Properties * |Select Name,DisplayName,DistinguishedName,rightsGuid| ?{$_.rightsGuid -eq $guid} | fl


Name              : User-Account-Restrictions
DisplayName       : Account Restrictions
DistinguishedName : CN=User-Account-Restrictions,CN=Extended-Rights,CN=Configuration,DC=securityeagles,DC=local
rightsGuid        : 4c164200-20c0-11d0-a768-00aa006e0529
```

- The GUID value for this specific example corresponds to the permission named User-Account-Restrictions.
- It is crucial to master this ACL enumeration methodology because this reconnaissance step is fundamental to identifying the misconfigured permissions that we will exploit later in the course

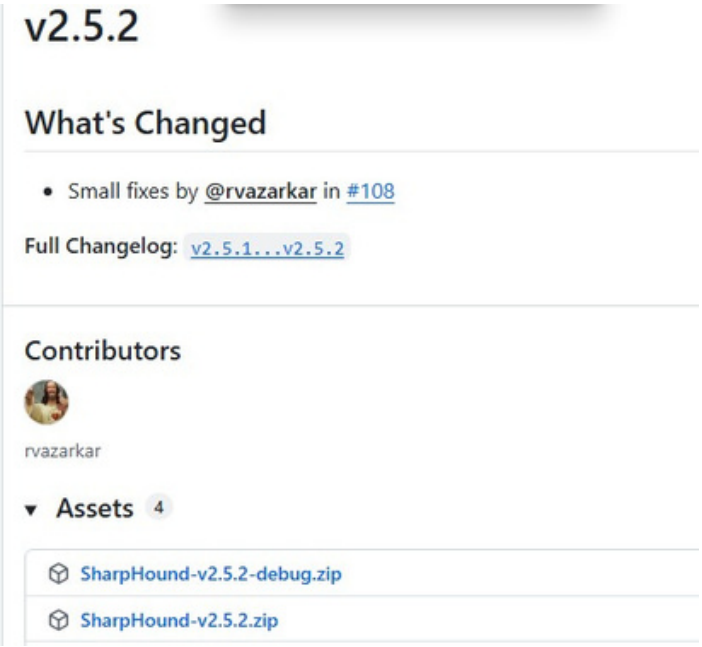
5- BloodHound mapping & queries

- While manual enumeration using PowerShell is crucial, quickly identifying complex privilege escalation and lateral movement paths within a large domain requires automated visualization .
- BloodHound is the industry-standard tool for mapping the hidden relationships and attack paths in an Active Directory environment.
- To analyze the domain, BloodHound requires data collected from the target network. Therefore, we must first download the SharpHound binary, which is the data collector tool for BloodHound.



source: https://github.com/SpecterOps/SharpHound

- The executable version of the data collector can be downloaded directly from the project's Releases page.



- Once the executable is downloaded, the next step is to transfer it to our host (EAGLES01 in mycase).

- To generate the files that BloodHound can analyze, SharpHound must query the Domain Controller using the Lightweight Directory Access Protocol (LDAP). This query extracts all necessary security relationships (users, groups, computers, ACLs, etc.).
- This is the command that SharpHound uses to collect LDAP data from the domain.

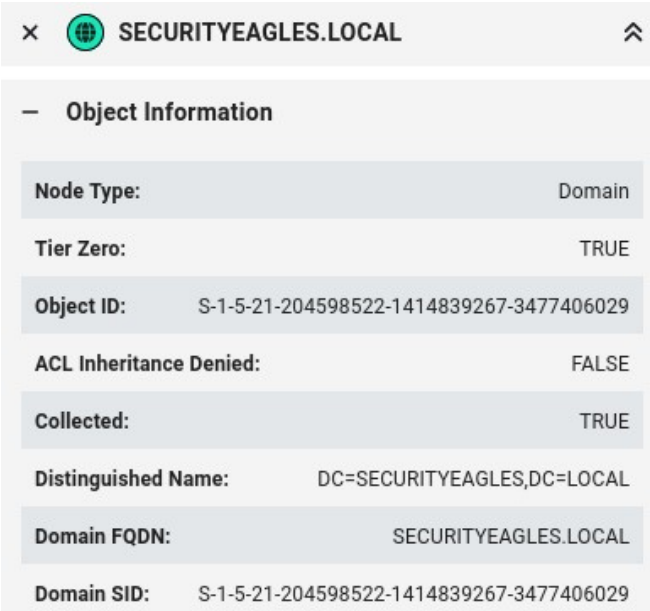
```
.\SharpHound.exe -c All --zipfilename DATA_securityEagles
```

```
PS C:\Users\m.dynamo\Downloads\SharpHound-v2.5.2> .\SharpHound.exe -c All --zipfilename DATA_securityEagles
2025-11-06T10:46:06.6818439+00:00|INFORMATION|This version of SharpHound is compatible with the 5.0.0 Release of B
2025-11-06T10:46:07.1767886+00:00|INFORMATION|Resolved Collection Methods: Group, LocalAdmin, GPOLocalGroup, Sessi
r, RDP, ObjectProps, DCOM, SPNTargets, PSRemote, UserRights, CARegistry, DCRegistry, CertServices
2025-11-06T10:46:07.2536868+00:00|INFORMATION|Initializing SharpHound at 10:46 on 06/11/2025
2025-11-06T10:46:07.5419203+00:00|INFORMATION|Resolved current domain to securityeagles.local
2025-11-06T10:46:07.9899579+00:00|INFORMATION|Flags: Group, LocalAdmin, GPOLocalGroup, Session, LoggedOn, Trusts,
COM, SPNTargets, PSRemote, UserRights, CARegistry, DCRegistry, CertServices
2025-11-06T10:46:08.2216463+00:00|INFORMATION|Beginning LDAP search for securityeagles.local
2025-11-06T10:46:08.4068337+00:00|INFORMATION|Beginning LDAP search for securityeagles.local Configuration NC
2025-11-06T10:46:08.4589311+00:00|INFORMATION|Producer has finished, closing LDAP channel
2025-11-06T10:46:08.4758191+00:00|INFORMATION|LDAP channel closed, waiting for consumers
2025-11-06T10:46:10.3617621+00:00|INFORMATION|Consumers finished, closing output channel
2025-11-06T10:46:10.3938347+00:00|INFORMATION|Output channel closed, waiting for output task to complete
Closing writers
2025-11-06T10:46:10.6215539+00:00|INFORMATION|Status: 307 objects finished (+307 153.5)/s -- Using 42 MB RAM
2025-11-06T10:46:10.6226454+00:00|INFORMATION|Enumeration finished in 00:00:02.4263518
2025-11-06T10:46:10.8512491+00:00|INFORMATION|Saving cache with stats: 17 ID to type mappings.
 2 name to SID mappings.
 2 machine sid mappings
```

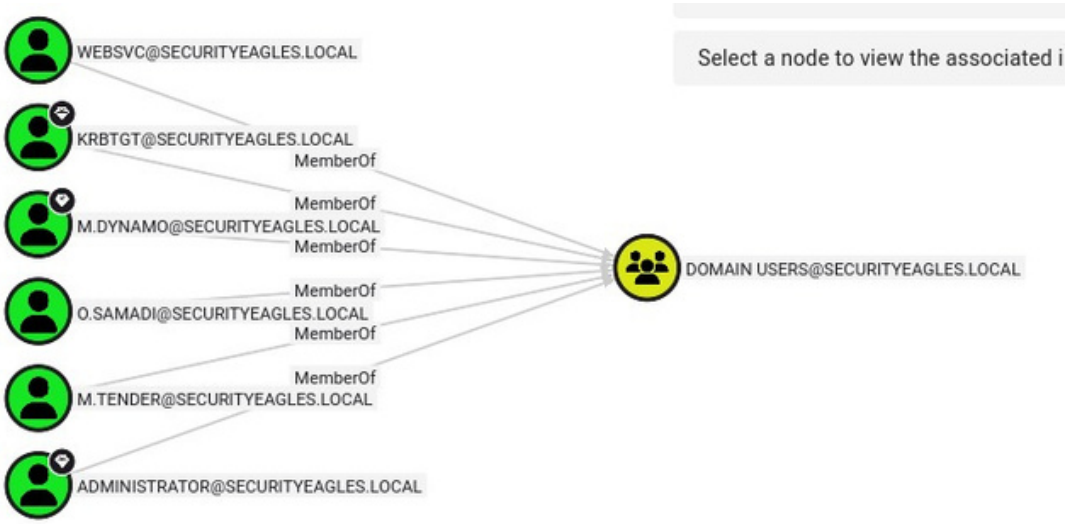
- Executing this command will produce a zipped archive of JSON files containing all raw enumeration data. The next step is to securely transfer these generated JSON files from the compromised host back to our attack machine for analysis in the BloodHound GUI.

- Once the collected JSON files are successfully uploaded, BloodHound generates a complex graph of the entire domain structure.
- Navigating this dense network requires efficiency to quickly find the objects relevant to our attack.
- we will frequently utilize the search bar to quickly locate specific users, groups, or machines for analysis.
- We first search for our domain, securityeagles.local, in the BloodHound interface.

- In the BloodHound GUI, every object (user, computer, group, or domain) is represented by a node. Interacting with these nodes is the primary way to gather intelligence and understand relationships.
- Upon clicking the domain node (represented by a circle), a detailed panel appears, displaying extensive information and attributes related to the domain object.
- This information panel is crucial because it provides details like the Domain SID, distinguished name, and security properties.



- BloodHound allows for rapid enumeration of all Domain Users
- We search for the 'Domain Users' group, which returns a complete list of all standard user accounts within the domain.

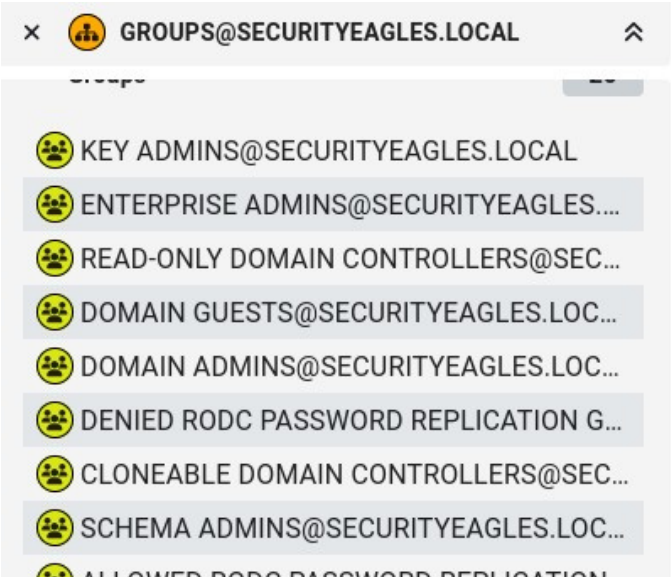


- This capability allows us to visualize the connections between low-privileged users and any critical resources they might have access to

- We can select a user object as an example to view all of that user's attributes, such as Node Type and Object ID.

- Understanding a user's basic metadata is the foundation for analyzing their attack path potential.

- In Active Directory, groups are the primary mechanism for assigning permissions and defining security boundaries.
- We can list and search for all group objects within the domain.



- Locating Domain Controllers is essential because they are the highest-value targets for compromise (e.g., for DCSync attacks) .
- We can also search for and visualize all “Domain Controllers” operating within the domain.

- "The Domain Admins group contains the most privileged domain users, which, in our lab, includes the default Administrator account and the custom account m.dynamo.
- Compromising any member of the Domain Admins group typically results in immediate domain compromise, often via techniques like DCSync , which we will dive into it in future sections

5- GPO discovery & settings audit

- Group Policy Objects are central to managing security settings, user environments, and software deployment across the entire domain. Analyzing GPOs is crucial for finding configuration errors that can be exploited .
- The Get-DomainGPO command retrieves a list of all configured Group Policy Objects (GPOs) within the domain.

```
Get-DomainGPO -credential $Cred
```

```
PS C:\Users\m.dynamo\Downloads> Get-DomainGPO -Credential $Cred


usncreated               : 5672
systemflags              : -1946157056
displayname              : Default Domain Policy
gpcmachineextensionnames : [{35378EAC-683F-11D2-A89A-00C04FBBCFA2}{53D6AB1B-2488-11D1-A28C-
00C04FB94F17}][{827D319E-6EA
                           C-11D2-A4EA-00C04F79F83A}{803E14A0-B4FB-11D0-A0D0-00A0C90F574B}]
[{B1BE8D72-6EAC-11D2-A4EA-00
                           C04F79F83A}{53D6AB1B-2488-11D1-A28C-00C04FB94F17}]
whenchanged              : 03/11/2025 15:58:23
objectclass              : {top, container, groupPolicyContainer}
```

- While viewing all GPOs is informative, a more targeted approach is often needed to understand the effective security policy on a single critical machine.
- We can refine the query to specify a particular host and list only the GPOs that are actively linked to it

```
Get-DomainGPO -Credential $Cred -ComputerIdentity EAGLES01
```

```
PS C:\Users\m.dynamo\Downloads> Get-DomainGPO -Credential $Cred -ComputerIdentity EAGLES01


usncreated               : 16483
displayname              : Disable Windows Defender
gpcmachineextensionnames : [{35378EAC-683F-11D2-A89A-00C04FBBCFA2}{D02B1F72-3407-48AE-BA88
                           -E8213C6761F1}]
whenchanged              : 03/11/2025 20:53:18
objectclass              : {top, container, groupPolicyContainer}
gpcfunctionalityversion  : 2
showinadvancedviewonly   : True
usnchanged               : 16490
dscorepropagationdata    : 01/01/1601 00:00:00
```

- Identifying host-specific GPOs allows us to pinpoint the exact policies that govern a target server's security configurations, user rights, and local administrative groups.

- In Active Directory, the structure is managed by Organizational Units, which are containers used to organize objects (users, computers, groups) and apply Group Policies.
- The Get-DomainOU command retrieves a list of all Organizational Units (OUs) configured within the domain.

```
Get-DomainOU -Credential $Cred
```

```
PS C:\Users\m.dynamo\Downloads> Get-DomainOU -Credential $Cred


usncreated              : 5804
systemflags             : -1946157056
iscriticalsystemobject  : True
gplink                  : [LDAP://CN={6AC1786C-016F-11D2-945F-00C04fB984F9},CN=Policies,CN=
                          System,DC=securityeagles,DC=local;0]
whenchanged             : 03/11/2025 14:56:05
objectclass             : {top, organizationalUnit}
showinadvancedviewonly  : False
usnchanged              : 5804
dscorepropagationdata   : {03/11/2025 20:30:33, 03/11/2025 15:35:44, 01/01/1601 00:04:16}
```

- Here we see the 'Groups' OU, which was configured from the start to contain all group objects within the domain.

```
whencreated             : 03/11/2025 20:30:33
instancetype            : 4
objectcategory          : CN=Organizational-Unit,CN=Schema,CN=Configuration,DC=securityeagle
                          s,DC=local
ou                      : Groups
objectguid              : 4eab1501-2dc1-4f29-b7d6-b2496f7ab2bb
whenchanged             : 03/11/2025 20:30:33
name                    : Groups
distinguishedname       : OU=Groups,DC=securityeagles,DC=local
usnchanged              : 16397
objectclass             : {top, organizationalUnit}
```

- This command returns a complete list of OUs, revealing the structural hierarchy of the domain.

```
Get-NetOU -Credential $Cred
```

```
PS C:\Users\m.dynamo\Downloads> Get-NetOU -Credential $Cred


usncreated              : 5804
systemflags             : -1946157056
iscriticalsystemobject  : True
gplink                  : [LDAP://CN={6AC1786C-016F-11D2-945F-00C04fB984F9},CN=Policies,CN=
                          System,DC=securityeagles,DC=local;0]
whenchanged             : 03/11/2025 14:56:05
objectclass             : {top, organizationalUnit}
showinadvancedviewonly  : False
usnchanged              : 5804
dscorepropagationdata   : {03/11/2025 20:30:33, 03/11/2025 15:35:44, 01/01/1601 00:04:16}
```

- We execute the Get-DomainGPO command, specifying the well-known GUID {6AC1786C-016F-11D2-945F-00C04fB984F9} found in the previous command using the -Identity parameter .

```
Get-DomainGPO -Identity "{6AC1786C-016F-11D2-945F-
00C04fB984F9}"-Credential $Cred
```

```
PS C:\Users\m.dynamo\Downloads> Get-DomainGPO -Identity "{6AC1786C-016F-11D2-945F-00C04fB98
4F9}" -Credential $Cred


usncreated               : 5675
systemflags              : -1946157056
displayname              : Default Domain Controllers Policy
gpcmachineextensionnames : [{827D319E-6EAC-11D2-A4EA-00C04F79F83A}{803E14A0-B4FB-11D0-A0D0
                           -00A0C90F574B}]
whenchanged              : 03/11/2025 14:56:05
objectclass              : {top, container, groupPolicyContainer}
gpcfunctionalityversion  : 2
```

6- Session & logon enumeration

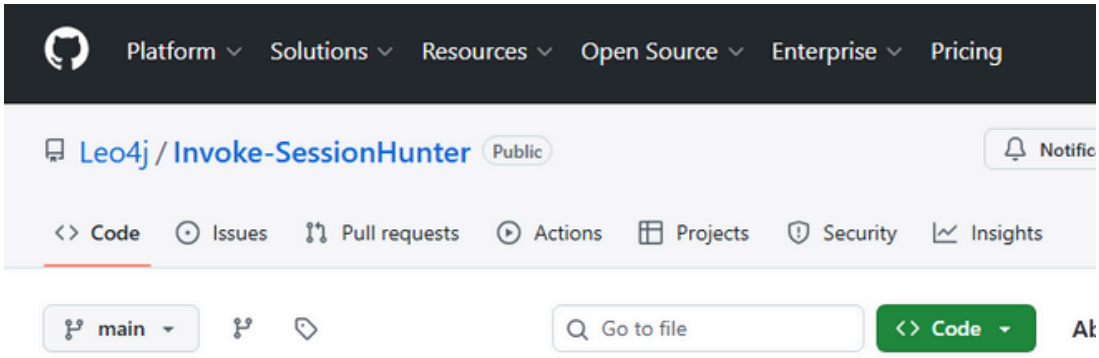
- The ability to move laterally within a domain is often predicated on having administrative rights on target hosts.
- So we check for Local Administrator Access for our current domain user across all domain computers.

```
Find-LocalAdminAccess -Verbose -Credential $Cred
```

```
PS C:\Users\m.dynamo\Downloads> Find-LocalAdminAccess -Verbose -Credential $Cred
VERBOSE: [Find-LocalAdminAccess] Querying computers in the domain
VERBOSE: [Get-Domain] Using alternate credentials for Get-Domain
VERBOSE: [Get-Domain] Extracted domain 'SECURITYEAGLES' from -Credential
VERBOSE: [Get-DomainSearcher] search base: LDAP://SECURITY-EAGLES-DC.securityeagles.loca
curityeagles,DC=local
VERBOSE: [Get-DomainSearcher] Using alternate credentials for LDAP connection
VERBOSE: [Get-DomainComputer] Get-DomainComputer filter string: (&(samAccountType=805306
VERBOSE: [Find-LocalAdminAccess] TargetComputers length: 3
WARNING: [Invoke-UserImpersonation] Executing LogonUser() with user: SECURITYEAGLES\m.dy
VERBOSE: [Invoke-UserImpersonation] Alternate credentials successfully impersonated
VERBOSE: [Find-LocalAdminAccess] Using threading with threads: 20
VERBOSE: [New-ThreadedFunction] Total number of hosts: 3
VERBOSE: [New-ThreadedFunction] Total number of threads/partitions: 3
VERBOSE: [New-ThreadedFunction] Threads executing
EAGLES01.securityeagles.local
VERBOSE: [New-ThreadedFunction] Waiting 100 seconds for final cleanup...
SECURITY-EAGLES-DC.securityeagles.local
VERBOSE: [New-ThreadedFunction] all threads completed
```

- In this case , our current user has administrative access on EAGLES01 and SECURITY-EAGLES-DC computers
- Identifying which machines the current user can access as a Local Administrator provides clear, immediate paths for lateral movement

- For extensive session enumeration, we will utilize the Invoke-SessionHunter cmdlet



- We will import the script and then execute the primary cmdlet, utilizing the -Domain parameter to ensure our reconnaissance targets the entire domain.

```
Invoke-SessionHunter -Domain securityeagles.local
```

```
PS C:\Users\m.dynamo\Downloads> Invoke-SessionHunter -Domain securityeagles.local
[+] Output saved to: C:\Users\m.dynamo\Downloads\SessionHunter.txt

[+] Elapsed time: 0:0:2.486
```

- the tool retrieves and displays information about active user sessions on remote computers and outputs the result in a txt file .

source: https://github.com/Leo4j/Invoke-SessionHunter

STAY TUNED FOR MORE

SECURITY EAGLES

www.securityeagles.com

FROM LEARNERS TO LEADERS